

Informe Técnico de Automatización QA

Prueba Técnica de Frontend — Jesús Agudelo

1. Repositorio del Proyecto

El código fuente y la suite de pruebas automatizadas se encuentran disponibles en el siguiente repositorio de GitHub: https://github.com/Vamkee/Prueba_QA_Front 😊

2. Preguntas y Respuestas Técnicas

A. Principales Desafíos Enfrentados

Uno de los principales desafíos fue trabajar con una aplicación SPA, ya que algunos elementos no cargaban inmediatamente y ciertos selectores antiguos ya no eran válidos. Tuve que mejorar los selectores, agregar esperas explícitas y asegurar que cada escenario iniciara con el navegador limpio. También encontré problemas al diligenciar los campos del checkout porque React no siempre reconocía el ingreso de datos de la forma tradicional. Además, tuve que controlar una alerta nativa de Chrome relacionada con contraseñas filtradas.

B. Técnicas de Pruebas y Enfoque de Automatización

Utilicé pruebas funcionales, positivas y negativas, aplicando BDD con Cucumber y escenarios escritos en Gherkin. La automatización se desarrolló con Selenium, Serenity BDD y el patrón Screenplay, separando las acciones, validaciones y datos de prueba. Se probaron funcionalidades como autenticación, catálogo, ordenamiento, carrito y checkout. También incluí esperas explícitas, validaciones de mensajes y capturas de pantalla después de cada paso.

C. Validación de la Experiencia del Usuario

Validé que los flujos se comportaran como lo haría un usuario real: iniciar sesión, consultar productos, agregarlos o eliminarlos del carrito, diligenciar el formulario y finalizar la compra. Verifiqué que los botones fueran visibles y funcionales, que los contadores y mensajes cambiaran correctamente y que los errores aparecieran cuando correspondía. Las capturas de pantalla y el reporte de Serenity me permitieron revisar visualmente cada paso y detectar posibles problemas de interacción.

D. Aspectos más Relevantes e Interesantes

Lo que más me llamó la atención fue resolver el problema del checkout. Aunque los campos parecían recibir el texto, la aplicación no siempre actualizaba correctamente su estado interno y por eso no permitía continuar. Fue necesario utilizar JavaScript junto con los eventos adecuados para que React reconociera los valores. También me pareció muy relevante implementar el patrón Screenplay, porque permitió organizar mejor el código y hacerlo más reutilizable y fácil de mantener.